

Давайте вернемся к yt-dlp. Он работает, да, но довольно утомительно получать загруженные видео на хост-машину.

Мы можем использовать Docker [тома](#), чтобы упростить хранение загрузок вне эфемерного хранилища контейнера. С помощью [bind mount](#) мы можем смонтировать файл или директорию с нашей собственной машины (хост-машины) в контейнер.

Давайте запустим контейнер с опцией `-v`, которая требует абсолютный путь. Мы монтируем нашу текущую папку как `/mydir` в нашем контейнере, перезаписывая все, что мы поместили в эту папку в нашем Dockerfile.

```
$ docker run -v "$(pwd):/mydir" yt-dlp https://www.youtube.com/watch?v=DptFY_MszQs
```

Итак, том — это просто папка (или файл), которая совместно используется между хост-машиной и контейнером. Если файл в томе изменяется программой, работающей внутри контейнера, изменения также сохраняются от уничтожения при завершении работы контейнера, так как файл существует на хост-машине. Это основное использование томов, так как в противном случае все файлы не были бы доступны при перезапуске контейнера. Тома также можно использовать для совместного использования файлов между контейнерами и запуска программ, способных загружать измененные файлы.

В нашем yt-dlp мы хотели смонтировать всю директорию, так как файлы имеют довольно случайные имена. Если мы хотим создать том только с одним файлом, мы могли бы сделать это, указав на него. Например, `-v "$(pwd)/material.md:/mydir/material.md"` таким образом мы могли бы редактировать material.md локально и иметь его изменения в контейнере (и наоборот). Обратите также внимание, что `-v` создает директорию, если файл не существует.

Упражнение 1.9

:::info Упражнение 1.9: Тома

В этом упражнении мы не будем создавать новый Dockerfile.

Образ `devopsdockeruh/simple-web-service` создает временную метку каждые две секунды в `/usr/src/app/text.log`, когда ему не дана команда. Запустите контейнер с `bind mount`, чтобы логи создавались в вашей файловой системе.

Отправьте команду, которую вы использовали для выполнения упражнения.

Подсказка: прочитайте заметку, которая была сделана прямо перед этим упражнением!

:::

Разрешение внешних подключений к контейнерам

Этот курс не предоставляет углубленного изучения механизмов межпрограммного взаимодействия. Если вы хотите изучить это подробнее, вам следует посмотреть курсы об Операционных Системах или Сетях. Здесь вам нужно знать только несколько простых вещей:

- Отправка сообщений: Программы могут отправлять сообщения на [URL](#) адреса, такие как этот: <http://127.0.0.1:3000>, где HTTP является [протоколом](#), 127.0.0.1 — это IP-адрес, а 3000 — [порт](#). Обратите внимание, что часть IP также может быть [hostname](#): 127.0.0.1 также называется [localhost](#), так что вместо этого вы можете использовать <http://localhost:3000>.

- Получение сообщений: Программам может быть назначено прослушивание любого доступного порта. Если программа прослушивает трафик на порту 3000, и сообщение отправлено на этот порт, программа получит и, возможно, обработает его.

Адрес `127.0.0.1` и `hostname localhost` являются специальными, они относятся к самой машине или контейнеру, так что если вы находитесь в контейнере и отправляете сообщение на `localhost`, целью является тот же контейнер. Аналогично, если вы отправляете запрос извне контейнера на `localhost`, целью является ваша машина.

Возможно **сопоставить порт вашей хост-машины с портом контейнера**. Например, если вы сопоставите порт 1000 на вашей хост-машине с портом 2000 в контейнере, а затем отправите сообщение на <http://localhost:1000> на вашем компьютере, контейнер получит это сообщение, если он прослушивает свой порт 2000.

Открытие соединения из внешнего мира в Docker контейнер происходит в два этапа:

- Открытие порта
- Публикация порта

Открытие порта контейнера означает сообщение Docker, что контейнер прослушивает определенный порт. Это не делает много, кроме как помогает людям с конфигурацией.

Публикация порта означает, что Docker будет сопоставлять порты хоста с портами контейнера.

Чтобы открыть порт, добавьте строку `EXPOSE <port>` в ваш Dockerfile

Чтобы опубликовать порт, запустите контейнер с `-p <host-port>:<container-port>`

Если вы опустите порт хоста и укажете только порт контейнера, Docker автоматически выберет свободный порт в качестве порта хоста:

```
$ docker run -p 4567 app-in-port
```

Мы также могли бы ограничить соединения только определенным протоколом, например, UDP, добавив протокол в конце: `EXPOSE <port>/udp` и `-p <host-port>:<container-port>/udp`.

:::tip Напоминание о безопасности: Открытие двери в интернет

Поскольку мы открываем порт для приложения, любой из интернета может войти и получить доступ к тому, что вы запускаете.

Не открывайте бездумно любые порты — способ для злоумышленника проникнуть внутрь — это использование порта, который вы открыли для незащищенного сервера. Простой способ избежать этого — определить порт на стороне хоста следующим образом: `-p 127.0.0.1:3456:3000`. Это позволит только запросам с вашего компьютера через порт 3456 к порту приложения 3000, без доступа извне.

Краткий синтаксис, `-p 3456:3000`, даст тот же результат, что и `-p 0.0.0.0:3456:3000`, что действительно открывает порт для всех.

Обычно это не рискованно. Но в зависимости от приложения, это то, что вы должны учитывать!

...

Упражнение 1.10

:::info Упражнение 1.10: Открытые порты

В этом упражнении мы не будем создавать новый Dockerfile.

Образ `devopsdockeruh/simple-web-service` запустит веб-сервис на порту `8080`, когда ему дан аргумент "server". В [Упражнении 1.8](#) вы уже создали образ, который можно использовать для запуска веб-сервиса без каких-либо аргументов.

Используйте теперь флаг `-p` для доступа к содержимому с помощью вашего браузера. Вывод в ваш браузер должен быть примерно таким: `{ message: "You connected to the following path: ...`

Отправьте использованные вами команды для этого упражнения.

:::